SearchClient Heuristic Design and Implementation (A*/Greedy)
Date: February 15, 2026

Overview

This document explains the improved heuristic used for informed search in the SearchClient project,
including why it is effective for levels that require push and pull actions.

Why the Baseline Goal-Count Heuristic is Weak

The default goal-count heuristic only counts how many goals are currently unsatisfied.
That gives very little guidance because:
- A box 1 step from goal and a box 40 steps from goal are treated similarly.
- It ignores walls and maze structure.
- It does not capture that the agent must first travel to a box before pushing/pulling it.

Improved Heuristic (Implemented)

The implemented heuristic combines three lower-bound components:

1) Box-goal distance (wall-aware)
   For each unsatisfied box goal cell with letter X, find the closest box X using
   precomputed shortest-path distances that respect walls.

2) Agent-goal distance (wall-aware)
   For each agent goal (digit), add shortest-path distance from that agent to its goal cell.

3) Agent-to-nearest-unplaced-box distance
   For each agent, add distance to the nearest same-color unplaced box.
   This captures the fact that push/pull cannot start until the agent reaches a box.

Formal Definition

Let d*(u, v) be shortest-path distance between cells u and v in the static grid (walls only).

h(s) =
  sum over box goals g of min over matching boxes b in state s of d*(b, g)
+ sum over agent goals ga of d*(agent(a), ga)
+ sum over agents a of min over same-color unplaced boxes b of distance(agent(a), b)

In the implementation:
- The first two terms use precomputed BFS distance maps from each goal cell.
- The third term uses Manhattan distance for speed (still admissible as a lower bound).

Preprocessing Strategy

Instead of precomputing all-pairs distances (very expensive), the code precomputes BFS from
EACH GOAL CELL only.

This gives a strong trade-off:
- Much less memory and preprocessing time than all-pairs.
- Constant-time lookup per cell-goal pair during h(s).

- Better guidance than Manhattan-only and goal-count heuristics.

Why this is Good for Push/Pull

Push/pull domains are sensitive to agent positioning. A state can have boxes nea
r goals,
but still require many steps because the agent is far away.

The agent-to-box term improves this behavior significantly by penalizing states
where agents
are poorly positioned to manipulate boxes.

Admissibility Notes

The heuristic is designed as a lower bound:
- Shortest-path distances with walls are lower bounds on real movement cost.
- Agent-to-box distance is a necessary movement before box manipulation can happ
en.
- Therefore A* with this h remains admissible in spirit of lower-bound design.

Practical Strengths

- Wall-aware and robust in labyrinth-like levels.
- Works for both box goals and agent goals.
- Fast per-state evaluation after preprocessing.
- Better guidance than simple goal count and often better than pure Manhattan.

Practical Weaknesses

- Ignores dynamic blocking interactions between boxes.
- Does not enforce one-to-one optimal box-goal assignment (unless extended with
matching).

Possible Next Improvements
----
1) Replace nearest-box-per-goal with Hungarian assignment per letter type.
2) Add deadlock penalties/checks for obvious irreversible box placements.
3) Use wall-aware distance also for agent-to-box term (instead of Manhattan) if
needed.

Where this is implemented

- searchclient_java/searchclient/Heuristic.java

How to run

Example:
java -jar ../server.jar -l ../levels/SAsoko1_04.lvl -c "java searchclient.Search
Client -greedy" -g -s 150 -t 180

Summary

For this assignment, the implemented heuristic is a strong and practical choice:
- More informative than goal count.
- Efficient due to goal-based BFS preprocessing.
- Better aligned with push/pull constraints via agent-to-box term.